

ПОДРОБНАЯ ЛЕКЦИЯ-МЕТОДИЧКА

с программными примерами на Python

Темы: единый подход; иерархическое обучение с подкреплением; определение суб-компонент; алгоритмы MAXQ и ALISP; мультиагентное обучение с подкреплением; методы полной и не прямой координации

Документ построен как учебная методичка: сначала дается теоретическая рамка, затем подробно разбираются два класса алгоритмов — иерархические и мультиагентные, после чего приводятся программные примеры на Python и набор заданий .

1. Единый подход к сложным задачам обучения с подкреплением

Единый подход означает, что сложную задачу принятия решений выгодно рассматривать как **композицию повторяющихся управленческих шаблонов**. Вместо того чтобы сразу учить одну большую политику на всех состояниях и действиях, исследователь выделяет уровни управления: стратегический, тактический и исполнительный.

- На стратегическом уровне выбирается общая цель или подцель: например, добраться до комнаты, захватить объект, занять выгодную позицию.
- На тактическом уровне определяется способ достижения подцели: каким маршрутом двигаться, кого назначить исполнителем, какой ресурс задействовать.
- На исполнительном уровне реализуются примитивные действия: шаг влево, открыть дверь, взять ключ, отправить сообщение второму агенту.

Такой взгляд естественно приводит к иерархии подзадач. Он же полезен и в мультиагентном случае: если несколько агентов работают в общей среде, их поведение также можно организовать по уровням — например, общий командный план и локальные навыки каждого агента.

2. Иерархическое обучение с подкреплением

Иерархическое обучение с подкреплением (Hierarchical Reinforcement Learning, HRL) — это семейство подходов, где агент учится не только примитивным действиям, но и **субзадачам, опциям, навыкам, макро-действиям**. Каждая субзадача имеет свою локальную цель, условия завершения и иногда собственную функцию вознаграждения.

- Переиспользование навыков: один и тот же навык можно вызывать из разных контекстов.
- Уменьшение глубины поиска: вместо длинной цепочки примитивных шагов выбирается высокоуровневая команда.
- Лучшая интерпретируемость: можно объяснить, какую подцель агент решал на каждом этапе.

- Ускорение обучения в больших средах за счет структурирования пространства решений.

3. Определение суб-компонент

Определение суб-компонент — ключевая инженерная задача в HRL. Под суб-компонентой понимается логически завершённый фрагмент поведения, который можно многократно использовать. Хорошая суб-компонента должна иметь ясную цель, ограниченную область применимости и понятный критерий завершения.

1. Выделить повторяющийся фрагмент поведения. Пример: навигация к позиции, поиск ресурса, проверка условия.
2. Определить входы подзадачи: какие признаки состояния нужны именно этой подзадаче.
3. Определить завершение: при каком условии подзадача считается выполненной.
4. Определить локальную награду: как поощрять быстрое и правильное завершение.
5. Проверить, можно ли переиспользовать подзадачу в нескольких сценариях.

Например, в задаче «агент должен взять ключ и открыть дверь» естественные суб-компоненты таковы: `NavigateToKey`, `PickKey`, `NavigateToDoor`, `OpenDoor`. Каждая из них проще полной задачи и имеет собственную завершённую семантику.

4. Алгоритм MAXQ

MAXQ — один из классических формализмов HRL. Его центральная идея состоит в том, что сложная функция ценности может быть разложена по дереву задач: **значение родительской задачи = значение выбранной подзадачи + ожидаемая стоимость продолжения после завершения подзадачи**.

- Корневая задача описывает полную цель агента.
- Внутренние узлы дерева — составные задачи.
- Листья — примитивные действия среды.
- Для каждой составной задачи можно хранить локальные оценки выгоды выбора подзадач.

Практический плюс MAXQ в том, что иерархия явно задаёт структуру решения. Обучение становится ближе к человеческой декомпозиции: «сначала доберись до области», «потом найди объект», «потом выполни финальное действие».

5. Алгоритм ALISP

ALISP (Approximate Lisp / программно-ориентированное описание частично заданной политики) использует идею, что исследователь может заранее задать **каркас поведения** в виде программы или сценария, а обучение будет заполнять

неопределенные места. Иначе говоря, мы не учим поведение с нуля, а ограничиваем поиск разумными программными шаблонами.

- Сильная сторона ALISP — возможность использовать предметные знания о порядке шагов.
- ALISP полезен там, где недопустим полный произвол агента и нужны безопасные или логичные последовательности действий.
- Ограничение поиска часто резко снижает сложность обучения.

Пример: известно, что дверь нельзя открыть без ключа. Тогда в ALISP-каркасе можно явно запретить ветку `open_door`, пока флаг `has_key` не стал истинным. Алгоритм по-прежнему может учить детали навигации, но не тратит время на бессмысленные или недопустимые действия.

6. Мультиагентное обучение с подкреплением

Мультиагентное RL изучает ситуации, где несколько агентов одновременно обучаются или действуют в общей среде. Это ведет к особой трудности: среда для одного агента становится **нестационарной**, потому что остальные агенты тоже меняют свои стратегии.

- Кооперативный режим: агенты максимизируют общую награду.
- Конкурентный режим: выигрыш одного может означать проигрыш другого.
- Смешанный режим: часть интересов общая, часть конфликтует.

7. Методы полной координации

Полная координация предполагает, что агенты могут строить совместный план, обмениваться информацией и, в пределах, выбирать совместное действие как единый вектор `joint-action`. В этом случае система приближается к централизованному управлению с распределенным исполнением.

- Централизованное обучение с децентрализованным исполнением.
- Совместная таблица $Q(a_1, a_2, \dots, a_n)$.
- Общий критик или общая функция ценности.
- Явный обмен сообщениями и синхронизация ролей.

8. Методы не прямой координации

Непрямая координация означает, что агенты согласуют действия не через постоянный обмен полными планами, а через изменение среды или через локальные правила. Это похоже на стигмерию: один агент оставляет след, другой его читает; один агент занимает ресурс, другой учитывает занятость.

- Маркировка клеток или маршрутов; феромонные карты; очереди задач.
- Локальные правила приоритета и избегания конфликтов.
- Распределение ролей по состоянию среды, а не по прямым командам.

Сравнение подходов

Подход	Идея	Сильные стороны	Ограничения	Когда применять
Обычное RL	Учить плоскую политику или Q-значения на уровне примитивных действий	Простота; мало инженерных решений	Плохо масштабируется	Небольшие среды и учебные задачи
Иерархическое RL	Разбить задачу на подзадачи и учить их отдельно	Повторное использование навыков; ускорение обучения	Нужна декомпозиция	Сложные многошаговые процессы
MAXQ	Разложить функцию ценности по дереву задач	Строгая математическая интерпретация	Нужно определить иерархию	Когда важен формальный анализ
ALISP	Добавить к поиску решения программные ограничения и сценарий	Сильное использование предметных знаний	Менее универсален	Когда есть четкий сценарный шаблон
Мультиагентное RL	Несколько обучающихся агентов в общей среде	Подходит для распределенных систем	Нестационарность и координация	Роботы, сеть, логистика, игры

9. Программный пример 1: учебная иерархическая задача в стиле MAXQ

Ниже показан учебный скрипт на Python. Он не претендует на полноту формализма MAXQ, но демонстрирует его дух: задача decomposes на подзадачи GetKey и OpenDoor, а навигация оформлена как переиспользуемый навык. Такой пример хорош для учебной аудитории, потому что видна логика разбиения и поток вызовов подзадач.

```
import random

class KeyDoorGrid:
    # Простая среда: агент должен взять ключ, затем открыть дверь.
    def __init__(self, size=5):
        self.size = size
        self.key_pos = (0, 4)
        self.door_pos = (4, 4)
        self.start_pos = (4, 0)
        self.reset()

    def reset(self):
        # Начальное состояние: агент в стартовой клетке, ключа нет, дверь закрыта.
        self.agent = self.start_pos
        self.has_key = False
```

```

        self.done = False
        return self.state()

def state(self):
    # Состояние включает позицию агента и флаг наличия ключа.
    return (self.agent[0], self.agent[1], int(self.has_key))

def move(self, action):
    # Примитивное действие: шаг в одном из четырех направлений.
    r, c = self.agent
    if action == 'up':
        r = max(0, r - 1)
    elif action == 'down':
        r = min(self.size - 1, r + 1)
    elif action == 'left':
        c = max(0, c - 1)
    elif action == 'right':
        c = min(self.size - 1, c + 1)
    self.agent = (r, c)
    reward = -1 # Небольшой штраф мотивирует решать задачу быстрее.

    if self.agent == self.key_pos:
        self.has_key = True
    if self.agent == self.door_pos and self.has_key:
        reward = 50
        self.done = True
    return self.state(), reward, self.done

def navigate(env, target):
    # Навигационный навык: двигаемся к целевой клетке жадно по Манхэттенскому
    # расстоянию.
    # В реальном MAXQ здесь могла бы обучаться отдельная подполитика.
    total_reward = 0
    while env.agent != target and not env.done:
        ar, ac = env.agent
        tr, tc = target
        if ar < tr:
            action = 'down'
        elif ar > tr:
            action = 'up'
        elif ac < tc:
            action = 'right'
        else:
            action = 'left'
        _, reward, _ = env.move(action)
        total_reward += reward
    return total_reward

def task_get_key(env):
    # Составная подзадача: добраться до ключа.
    return navigate(env, env.key_pos)

def task_open_door(env):

```

```

# Составная подзадача: добраться до двери и завершить эпизод.
return navigate(env, env.door_pos)

def solve_episode(env):
    # Корневая задача: если ключа нет, сначала взять ключ; потом открыть дверь.
    env.reset()
    reward_sum = 0
    if not env.has_key:
        reward_sum += task_get_key(env)
    reward_sum += task_open_door(env)
    return reward_sum, env.done

```

Комментарий к примеру. Здесь роль иерархии видна явно: `solve_episode` — корневая задача; `task_get_key` и `task_open_door` — подзадачи; `move` — примитивный уровень. Навигация вынесена в отдельный навык `navigate`, что позволяет повторно использовать ее для ключа и двери.

10. Программный пример 2: ALISP-подобное ограничение сценария

Следующий пример показывает идею ALISP: мы не разрешаем выполнять действие открытия двери, пока агент не получил ключ. Иными словами, мы встроили в систему знаний предметное правило. Это резко уменьшает пространство ошибочных сценариев.

```

class SafeKeyDoorPolicy:
    # Программный каркас поведения с ограничениями.
    def choose_high_level_action(self, state):
        row, col, has_key = state

        # Если ключа нет, сценарий требует сначала получить ключ.
        if not has_key:
            return 'go_to_key'

        # Только после получения ключа допускается движение к двери.
        return 'go_to_door'

def run_alisp_like_episode(env, policy):
    state = env.reset()
    total_reward = 0

    while not env.done:
        high_level = policy.choose_high_level_action(state)

        if high_level == 'go_to_key':
            total_reward += navigate(env, env.key_pos)
        elif high_level == 'go_to_door':
            total_reward += navigate(env, env.door_pos)

        state = env.state()

```

```
return total_reward
```

Важная мысль: ALISP не обязателен как конкретный синтаксис языка. Для учебных целей достаточно понять принцип — часть решений задается программно, а обучение или поиск работают внутри разрешенного каркаса.

11. Программный пример 3: кооперативное мультиагентное обучение

В этом примере два агента должны встретиться в одной клетке. Награда выдается только тогда, когда они скоординированно приходят к общей цели. Это минималистичная кооперативная задача, показывающая различие между полной и непрямой координацией.

```
import random
from collections import defaultdict

ACTIONS = ['up', 'down', 'left', 'right', 'stay']

class MeetingWorld:
    # Два агента перемещаются по полю 4x4 и должны встретиться в целевой клетке.
    def __init__(self, size=4, goal=(2, 2)):
        self.size = size
        self.goal = goal
        self.reset()

    def reset(self):
        self.a1 = (0, 0)
        self.a2 = (3, 3)
        return self.state()

    def state(self):
        return (self.a1, self.a2)

    def _apply(self, pos, action):
        r, c = pos
        if action == 'up':
            r = max(0, r - 1)
        elif action == 'down':
            r = min(self.size - 1, r + 1)
        elif action == 'left':
            c = max(0, c - 1)
        elif action == 'right':
            c = min(self.size - 1, c + 1)
        return (r, c)

    def step(self, action1, action2):
        self.a1 = self._apply(self.a1, action1)
        self.a2 = self._apply(self.a2, action2)
        reward = -1
        done = False
        if self.a1 == self.goal and self.a2 == self.goal:
```

```

        reward = 30
        done = True
        return self.state(), reward, done

def epsilon_greedy_joint(Q, state, epsilon=0.2):
    # При полной координации можно выбрать совместное действие сразу для двух
    # агентов.
    joint_actions = [(a1, a2) for a1 in ACTIONS for a2 in ACTIONS]
    if random.random() < epsilon:
        return random.choice(joint_actions)
    return max(joint_actions, key=lambda act: Q[(state, act)])

def train_joint_q(epsisodes=200):
    env = MeetingWorld()
    Q = defaultdict(float)
    alpha, gamma, epsilon = 0.2, 0.95, 0.2

    for _ in range(epsisodes):
        state = env.reset()
        done = False
        while not done:
            joint_action = epsilon_greedy_joint(Q, state, epsilon)
            next_state, reward, done = env.step(*joint_action)
            best_next = max(Q[(next_state, act)] for act in [(a1, a2) for a1 in
ACTIONS for a2 in ACTIONS])
            Q[(state, joint_action)] += alpha * (reward + gamma * best_next -
Q[(state, joint_action)])
            state = next_state
        return Q

```

Если мы хотим моделировать непрямую координацию, то вместо joint-action можно позволить агентам выбирать действия независимо, но читать общую карту следов или учитывать занятость клеток. Тогда координация происходит через состояние среды, а не через полное совместное планирование.

12. Методические рекомендации по разбору примеров

- Сначала попросите студентов назвать уровни управления: цель, подцели, примитивные действия.
- Затем выделите признаки состояния, необходимые каждой подзадаче.
- После этого обсудите, где именно экономится поиск: за счет иерархии, за счет программных ограничений, за счет координации.
- Отдельно сравните интерпретируемость результатов: дерево подзадач обычно легче объяснить, чем одну большую таблицу Q.

20 заданий

Задания ориентированы на самостоятельную доработку представленных программных примеров, сравнительный анализ алгоритмов и проектирование иерархий навыков.

№	Формулировка задания	Ожидаемый результат
1	Измените размер GridWorld в примере MAXQ с 5×5 на 7×7 и проанализируйте, как изменилось число шагов до цели.	Код, краткий отчет, интерпретация результатов обучения.
2	Добавьте в GridWorld две дополнительные стены и сравните среднее вознаграждение до и после изменения.	Код, краткий отчет, интерпретация результатов обучения.
3	Реализуйте для примера MAXQ логирование числа вызовов подзадач Navigate и GetKey.	Код, краткий отчет, интерпретация результатов обучения.
4	Измените структуру иерархии: выделите отдельную подзадачу MoveToDoog и покажите, как это влияет на читаемость решения.	Код, краткий отчет, интерпретация результатов обучения.
5	Добавьте стохастичку: с вероятностью 0.1 агент выполняет случайное действие вместо выбранного.	Код, краткий отчет, интерпретация результатов обучения.
6	Сравните иерархическое решение и плоский Q-learning на одной и той же среде.	Код, краткий отчет, интерпретация результатов обучения.
7	Сформулируйте набор признаков состояния для складского робота и обоснуйте, какие из них относятся к локальным, а какие к глобальным.	Код, краткий отчет, интерпретация результатов обучения.
8	Постройте дерево подзадач для банкомата, интернет-магазина или робота-пылесоса.	Код, краткий отчет, интерпретация результатов обучения.
9	Реализуйте ограничение ALISP: запрет на выполнение действия open_door до получения ключа.	Код, краткий отчет, интерпретация результатов обучения.
10	Модифицируйте ALISP-пример так, чтобы ключ мог появляться в двух разных позициях.	Код, краткий отчет, интерпретация результатов обучения.
11	Добавьте в ALISP накопление штрафа за нарушение рекомендованного сценария и сравните результаты.	Код, краткий отчет, интерпретация результатов обучения.
12	Сделайте функцию, которая автоматически проверяет корректность иерархии:	Код, краткий отчет, интерпретация результатов обучения.

	достижимость всех конечных подцелей и отсутствие циклов в дереве.	
13	В кооперативной мультиагентной задаче увеличьте число агентов до трех и реализуйте поочередный выбор действий.	Код, краткий отчет, интерпретация результатов обучения.
14	Добавьте в кооперативную задачу обмен сообщениями между агентами длиной в один бит и проанализируйте выигрыш.	Код, краткий отчет, интерпретация результатов обучения.
15	Реализуйте централизованный выбор joint-action для двух агентов и сравните с независимым Q-learning.	Код, краткий отчет, интерпретация результатов обучения.
16	Добавьте штраф за столкновение агентов и исследуйте, как меняется политика.	Код, краткий отчет, интерпретация результатов обучения.
17	Смоделируйте непрямую координацию: агенты выбирают маршрут, ориентируясь на феромонную карту или доску следов.	Код, краткий отчет, интерпретация результатов обучения.
18	Проведите эксперимент по чувствительности к коэффициенту обучения α и коэффициенту дисконтирования γ .	Код, краткий отчет, интерпретация результатов обучения.
19	Подготовьте таблицу сравнения MAXQ, ALISP и независимого мультиагентного Q-learning по критериям: интерпретируемость, скорость обучения, сложность настройки.	Код, краткий отчет, интерпретация результатов обучения.
20	Разработайте собственную учебную задачу, где есть иерархия подзадач и минимум два взаимодействующих агента; опишите состояния, действия и награды.	Код, краткий отчет, интерпретация результатов обучения.

13. Контрольные вопросы

- Почему плоское обучение с подкреплением плохо масштабируется на сложных многошаговых задачах?
- Что такое суб-компонента в иерархическом обучении?

8. Чем MAXQ отличается от общего представления задачи в виде набора опций?
9. Какую роль играют предметные знания в ALISP?
10. Почему в мультиагентном обучении возникает нестационарность?
11. Чем полная координация отличается от не прямой координации?
12. Какие преимущества и недостатки дает централизованное обучение?
13. Почему повторное использование навыков ускоряет обучение?

14. Заключение

Иерархическое и мультиагентное обучение с подкреплением нужны тогда, когда задача уже не укладывается в плоскую схему «состояние — действие — награда». Единый подход состоит в том, чтобы искать структуру: выделять подцели, проектировать суб-компоненты, использовать предметные ограничения и организовывать координацию между агентами. Именно структура делает сложные задачи решаемыми, объяснимыми и удобными для программной реализации.